

Architekturdokumentation

WebLab Projekt -- Konzeption, Struktur und technische Architektur

Autor(en):	Vorname Nachname (vorname.nachname@stud.hslu.ch) – Matrikelnr.: XX-XXX-XXX
Modul:	WebLab (FS 2026)
Dozierende/Betreuung:	Dozent Name
Version:	1.0
Datum:	31.08.2026

Management Summary / Abstract

Dieses Dokument beschreibt die Softwarearchitektur der im Rahmen des Moduls WebLab entwickelten Webanwendung. Es orientiert sich an gängigen Architekturmustern (u. a. arc42) und dokumentiert die Systemziele, Randbedingungen, Bausteine, Datenstrukturen sowie zentrale Architekturentscheidungen (ADRs).

Inhaltsverzeichnis

1	Einführung & Ziele	2
1.1	Aufgabenstellung & Motivation	2
1.2	Qualitätsziele	2
1.3	Stakeholder & Zielgruppen	2
2	Randbedingungen & Technologiestack	3
2.1	Technische Rahmenbedingungen	3
2.2	Organisatorische & rechtliche Rahmenbedingungen	6
2.3	Konventionen & Richtlinien	6
3	Kontextabgrenzung	7
3.1	Fachlicher Kontext	7
3.2	Technischer Kontext	7
4	Bausteinsicht	8
4.1	Gesamtsystem (Ebene 1)	8
4.2	Frontend-Bausteine (Ebene 2)	8
4.3	Backend-Bausteine (Ebene 2)	8
5	Laufzeitsicht	8
5.1	Typischer Anwendungsfall: Benutzer-Authentifizierung	8
5.2	Datenabruf und State-Update	9
6	Verteilungssicht & Deployment	9
6.1	Infrastruktur-Übersicht	9
6.2	CI/CD-Pipeline	9
7	Datenmodell & Persistenz	10
7.1	Entitäten & Relationen	10
7.2	Datenmigration & Validierung	10
8	Querschnittskonzepte	10
8.1	Sicherheitskonzept (Security)	10
8.2	Fehlerbehandlung & Logging	11
8.3	UI/UX & Responsive Design	11
9	Architekturentscheidungen (ADRs)	11
9.1	Übersicht der Architekturentscheidungen	11

1 Einführung & Ziele

1.1 Aufgabenstellung & Motivation

TODO

Beschreiben Sie hier kurz die Problemstellung und die Vision des WebLab-Projekts. Was ist das Hauptziel der Webapplikation?

Die vorliegende Webanwendung wurde im Rahmen des Moduls **WebLab** an der Hochschule Luzern (Departement Informatik) entwickelt. Ziel ist es, eine modulare, performante und benutzerfreundliche Webanwendung zu realisieren, welche moderne Best Practices im Fullstack-Bereich demonstriert.

1.2 Qualitätsziele

Die zentralen Qualitätsziele orientieren sich an ISO/IEC 25010 und sind in folgender Priorität definiert:

Priorität	Qualitätsmerkmal	Konkretisierung / Messbarkeit
1	Benutzerfreundlichkeit (Usability)	Intuitive Bedienung, konsistentes UI/UX, Reaktionszeit bei Interaktionen < 100ms.
2	Wartbarkeit (Maintainability)	Saubere modulare Trennung (Clean Architecture / Komponenten), TypeScript Typisierung, Testabdeckung.
3	Zuverlässigkeit (Reliability)	Robuste Fehlerbehandlung im Frontend & Backend, Valide Datenvalidierung auf API-Ebene.
4	Performance	Optimierte Asset-Bundles, schnelles Initial Rendering (< 1.5s First Contentful Paint).
5	Sicherheit (Security)	Sichere Authentifizierung, Schutz vor XSS/CSRF, Input-Sanitizing.

1.3 Stakeholder & Zielgruppen

Rolle / Stakeholder	Erwartungshaltung	Be-deu-tung für das Sys-tem
Endbenutzer	Schnelle, selbsterklärende und fehlerfreie Nutzung der Kernfunktionen.	Di-rek-te In-

Rolle / Stakeholder	Erwartungshaltung	Be- deu- tung für das Sys- tem
		ter- ak- ti- on mit dem Front- end.
Dozierende / Experten	Technisch saubere Umsetzung, Einhaltung der WebLab-Vorgaben, Dokumentation.	Be- wer- tung & Ab- nah- me.
Entwickler / Betreiber	Gute Erweiterbarkeit, klare Code-Struktur, einfaches lokales Setup und Deployment.	War- tung und Wei- ter- ent- wick- lung.

2 Randbedingungen & Technologiestack

2.1 Technische Rahmenbedingungen

Die Entwicklung basiert auf modernen Webtechnologien und unterliegt folgenden Rahmenbedingungen:

Bereich	Technologie / Standard	Be-gründung / Kon-text
Frontend	TypeScript, Modernes Framework (z. B. Angular / React / Vue)	Mo-der-ne Kom-po-nen-ten-ar-chi-tek-tur, Typ-sicher-heit und Sta-te-Ma-nage-ment.
Styling	CSS3 / SCSS / Tailwind CSS	Re-spon-si-ve De-sign, Kon-sis-ten-tes De-sign Sys-tem.
Backend / API	Node.js / Express / NestJS / REST API	Asyn-chron-es I/O, naht-

Bereich	Technologie / Standard	Be- grün- dung / Kon- text
		lo- se Ty- pe- Script- In- tegra- ti- on mit Shared Ty- pes.
Datenbank / Storage	PostgreSQL / MongoDB / SQLite	Struk- turier- te Da- ten- spei- che- rung und re- latio- na- le bzw. do- ku- men- ten- ba- sier- te In- tegri- tät.
Versionskontrolle	Git & GitHub	Fea- ture- Branch-

Bereich	Technologie / Standard	Be-gründung / Kon-text
		Work-flow, Pull Re-quests und Code Re-views.
Dokumentation	Typst	Prä-zise, ver-sio-nier-ba-re und mo-der-ne PDF-Ge-ne-rie-rung.

2.2 Organisatorische & rechtliche Rahmenbedingungen

- **Lizenzen:** Einsatz ausschliesslich von Open-Source-Bibliotheken unter permissiven Lizenzen (MIT, Apache 2.0).
- **Datenschutz:** Es werden keine sensiblen Personendaten ohne explizite Einwilligung oder im Produktivbetrieb unverschlüsselt verarbeitet (DSG / DSGVO Konformität im Konzept).
- **Abgabetermin & Meilensteine:** Einhaltung der vorgegebenen Modul-Deadlines gemäss Web-Lab-Zeitplan.

2.3 Konventionen & Richtlinien

- **Code Style:** ESLint & Prettier für einheitliche Code-Formatierung.
- **Naming Conventions:** CamelCase für Variablen/Funktionen, PascalCase für Klassen/Komponenten, kebab-case für Dateinamen.
- **Git Commits:** Conventional Commits (feat: , fix: , docs: , refactor: , test: , chore:).

3 Kontextabgrenzung

3.1 Fachlicher Kontext

Der fachliche Kontext definiert die Systemgrenze aus Benutzersicht und beschreibt die externen Akteure sowie den Informationsaustausch.

i Hinweis

Fügen Sie bei Bedarf ein Kontextdiagramm (z. B. via Mermaid oder exportiertes Bild aus Doc/assets/) ein.

Nachbarsystem / Akteur	Eingabe in das System	Ausgabe aus dem System
Benutzer (Webbrowser)	Benutzerinteraktionen, Formulareingaben, Authentifizierungsdaten	Dynamische UI-Views, Bestätigungen, Fehlermeldungen
Dritte APIs (z. B. OAuth, Wetter, etc.)	API-Tokens, externe Daten-Payloads	Anfragen, Webhook-Trigger

3.2 Technischer Kontext

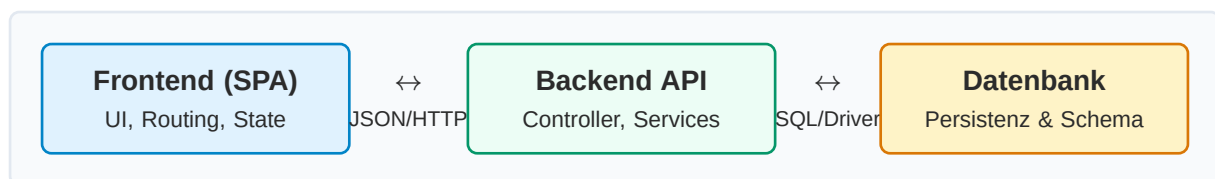
Der technische Kontext beschreibt die Kommunikationskanäle und Protokolle zwischen den Komponenten:

- **Frontend** ⇒ **Backend**: HTTP/HTTPS via RESTful JSON API (bzw. WebSockets für Echtzeitkommunikation).
- **Backend** ⇒ **Datenbank**: TCP/IP über standardisierte Datenbanktreiber / ORM mit Connection-Pooling.
- **Client-Plattformen**: Moderne Desktop- und Mobil-Browser (Chromium, Firefox, Safari) mit responsivem Viewport.

4 Bausteinsicht

4.1 Gesamtsystem (Ebene 1)

Das Gesamtsystem gliedert sich in eine klassische 3-Schichten-Architektur:



4.2 Frontend-Bausteine (Ebene 2)

Das Frontend ist nach Feature-Modulen und Shared-Komponenten modularisiert:

- **Core / App-Shell**: Grundlegendes Layout, Header, Navigation, globale Interceptoren und State-Services.
- **Feature-Module**: Kapselung von fachlichen Domänen (z. B. Authentifizierung, Dashboard, Datenansicht, Einstellungen).
- **Shared / UI-Components**: Wiederverwendbare atomare UI-Elemente (Buttons, Modals, Form Controls, Tables).
- **Services & API-Clients**: Datenabruf, Caching und Synchronisation mit dem Backend.

4.3 Backend-Bausteine (Ebene 2)

Das Backend folgt dem Schichtenmodell zur Trennung von Belangen (Separation of Concerns):

1. **Controller / Routing-Layer**: Entgegennahme von HTTP-Requests, Validierung der Payloads (DTOs) und HTTP-Response-Handling.
2. **Service- / Business-Logic-Layer**: Zentrale Geschäftslogik, Domain-Validierung und Orchestrierung.
3. **Data Access / Repository-Layer**: Abstraktion des Datenbankzugriffs, CRUD-Operationen und Transaktionsverwaltung.
4. **Middleware**: Authentifizierung (JWT Verification), Logging, CORS und globales Error-Handling.

5 Laufzeitsicht

5.1 Typischer Anwendungsfall: Benutzer-Authentifizierung

Der Ablauf für Login und Autorisierung gestaltet sich wie folgt:

1. Der Benutzer gibt Zugangsdaten im Anmeldeformular ein.
2. Das Frontend validiert das Formular lokal und sendet einen `POST /api/auth/login` Request an das Backend.
3. Der Backend-Auth-Controller validiert die Eingaben, hasht das Passwort und vergleicht den Hash mit dem Datenbankeintrag.

- Bei erfolgreicher Verifikation generiert das Backend ein signiertes JSON Web Token (JWT) und gibt dieses zurück.
- Das Frontend speichert den Token sicher (z. B. Secure HTTP-Only Cookie oder im Auth-State) und fügt ihn bei Folgeanfragen im `Authorization: Bearer <token>` Header ein.

5.2 Datenabruf und State-Update

Schritt	Beteiligte Komponente	Aktion / Beschreibung
1	UI-Komponente	Löst beim Rendern oder durch Benutzeraktion ein Event / Dispatch aus.
2	Data Service	Sendet einen HTTP GET Request an den zuständigen API-Endpunkt.
3	Backend API	Verifiziert den JWT-Token, prüft Berechtigungen und holt Daten via ORM/SQL.
4	Frontend Store	Aktualisiert den lokalen State mit der erhaltenen Response.
5	UI-Komponente	Reagiert reaktiv auf die Zustandsänderung und aktualisiert die Ansicht.

✓ Tipp

Sequenzdiagramme können direkt in Typst oder als Vektorgrafiken (`.svg`) in `Doc/assets/` abgelegt und eingebunden werden.

6 Verteilungssicht & Deployment

6.1 Infrastruktur-Übersicht

Die Webanwendung ist für containerisierte Umgebungen konzipiert und kann sowohl lokal als auch in Cloud-Infrastrukturen betrieben werden.

Knoten / Container	Umgebung	Beschreibung / Konfiguration
Frontend Static Host	Nginx / Cloudflare Pages / Vercel	Auslieferung kompilierter Produktions-Assets (HTML, JS, CSS, Media).
Backend Service	Node.js Container (Docker)	Containerisierter Backend-Prozess, REST API, Environment-Konfiguration via <code>.env</code> .
Database Instance	PostgreSQL / MongoDB	Persistentes Volume, automatische Migrationen bei Start.

6.2 CI/CD-Pipeline

Zur kontinuierlichen Qualitätssicherung ist eine automatisierte Pipeline (z. B. via GitHub Actions) vorgesehen:

1. **Linting & Type-Check:** Automatische Überprüfung via `npm run lint` und `tsc --noEmit`.
2. **Automatisierte Tests:** Ausführung von Unit- und Integrationstests (`npm test`).
3. **Build-Prozess:** Kompilierung des Frontends (`npm run build`) und Backends.
4. **Container Build & Registry:** Erstellung der Docker-Images und Tagging mit Git-Commit-Hash.

7 Datenmodell & Persistenz

7.1 Entitäten & Relationen

Das Datenmodell bildet die fachlichen Kernentitäten der Anwendung ab.

TODO

Passen Sie die nachfolgende Entitäten-Tabelle an Ihr konkretes Datenbankschema an.

Entität	Feld	Typ	Beschreibung / Constraints
User	id	UUID / Int	Primary Key, automatische Generierung
	email	String	Unique, Pflichtfeld, validierte E-Mail-Adresse
	passwordHash	String	Bcrypt- / Argon2-Hash, nicht exponiert
	role	Enum	Rolle (ADMIN , USER)
	createdAt	Timestamp	Erstellungszeitpunkt
EntityItem	id	UUID / Int	Primary Key
	userId	UUID / Int	Foreign Key → User(id)
	title	String(255)	Titel / Bezeichnung
	status	Enum	Status (DRAFT , PUBLISHED , ARCHIVED)
	updatedAt	Timestamp	Letztes Änderungsdatum

7.2 Datenmigration & Validierung

- **Schema-Migrationen:** Werden über ORM-Migrationstools (z. B. Prisma, TypeORM, Drizzle) versioniert und deterministisch ausgeführt.
- **Validierung:** Doppelte Validierung sowohl im Frontend (für unmittelbares Benutzerfeedback) als auch zwingend im Backend (via Schema-Validierung wie Zod / class-validator).

8 Querschnittskonzepte

8.1 Sicherheitskonzept (Security)

- **Authentifizierung & Autorisierung:** Token-basierte Authentifizierung (JWT) mit angemessener Expiration Time und Role-Based Access Control (RBAC).
- **Schutz gegen gängige Web-Schwachstellen (OWASP Top 10):**
 - **XSS:** Automatisches Escaping durch das Frontend-Framework, Content Security Policy (CSP).
 - **CSRF:** SameSite-Cookie-Flags und CSRF-Tokens für zustandsbehaftete Mutationen.

- **SQL-Injection:** Ausschliessliche Verwendung von parametrisierten Queries über ORM / Query Builder.
- **HTTPS & Verschlüsselung:** Durchgehende TLS-Verschlüsselung im Transit, Passwort-Hashing mit Argon2id / Bcrypt.

8.2 Fehlerbehandlung & Logging

- **Frontend:** Globaler Error-Boundary-Handler, benutzerfreundliche Toast-Nachrichten, Graceful Degradation bei Verbindungsunterbrüchen.
- **Backend:** Zentraler Exception-Filter / Middleware, strukturierte JSON-Logs (z. B. Pino / Winston) mit Request-IDs zur Nachverfolgbarkeit.
- **API-Fehlerformat:** Standardisierte RFC 7807 (Problem Details for HTTP APIs) Fehlerrückgaben:

```
{
  "statusCode": 400,
  "error": "Bad Request",
  "message": "Validierungsfehler im Feld 'email'",
  "timestamp": "2026-03-01T10:00:00.000Z"
}
```

8.3 UI/UX & Responsive Design

- Mobile-First-Ansatz mit Breakpoints für Mobile (< 640px), Tablet (640 – 1024px) und Desktop (> 1024px).
- Barrierefreiheit (a11y) gemäss WCAG 2.1 AA Kriterien (Farbkontraste, Tastaturbedienbarkeit, ARIA-Labels).

9 Architekturentscheidungen (ADRs)

In diesem Kapitel werden fundamentale Entwurfs- und Architekturentscheidungen dokumentiert und begründet.

9.1 Übersicht der Architekturentscheidungen

ADR-001: Wahl des Frontend-Frameworks und TypeScript Akzeptiert 15.02.2026

Kontext & Problemstellung:

Für die Entwicklung der WebLab-Anwendung wird ein modernes Frontend-Framework benötigt, welches strikte Typsicherheit, komponentenbasierte Wiederverwendbarkeit und ein starkes Ökosystem bietet.

Entscheidung:

Wir setzen auf ein modernes Single-Page-Application-Framework mit durchgängigem Einsatz von TypeScript.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Hohe Wartbarkeit, statische Fehlerprüfung zur Compile-Zeit, hervorragende IDE-Unterstützung und Autovervollständigung.
- **Nachteile / Trade-offs:** Geringfügiger Boilerplate-Overhead beim Erstellen von Interfaces und DTOs.

ADR-002: Authentifizierung via JWT (JSON Web Tokens) Akzeptiert 20.02.2026**Kontext & Problemstellung:**

Das System erfordert eine sichere, zustandslose Authentifizierung zwischen Frontend-SPA und Backend-API.

Entscheidung:

Einsatz von JSON Web Tokens (JWT) mit HMAC-SHA256 Signatur und kurzer Gültigkeitsdauer.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Stateless Backend (keine Server-Sessions in der DB erforderlich), gut skalierbar, einfache Handhabung im REST-Kontext.
- **Nachteile / Trade-offs:** Token-Revocation (Blacklisting) erfordert zusätzliche Mechanismen wie Refresh Tokens oder Redis-Cache.

ADR-003: Dokumentationserstellung mit Typst Akzeptiert 25.02.2026**Kontext & Problemstellung:**

Die geforderten Projektdokumente (Architektur, Reflexion, Journal) müssen im Team sauber versioniert, ansprechend formatiert und schnell als PDF exportiert werden können.

Entscheidung:

Nutzung von Typst anstelle von Word oder schwergewichtigem LaTeX.

Konsequenzen & Vor-/Nachteile:

- **Vorteile:** Sehr schnelle Kompilierung (< 100ms), saubere Code-basierte Struktur in Git, moderne Syntax, hervorragendes Typography-Layout.
- **Nachteile / Trade-offs:** Einarbeitung in Typst-Syntax für Teammitglieder nötig (wird durch Vorlagen stark vereinfacht).